
Traitement d'Images Distribué

Architecture Client / Serveur / Worker

HOUCHI Hocine

MERABET Islam

Rapport de projet réalisé dans le cadre du module Système d'Exploitation.

Résumé

Ce projet implémente une architecture distribuée pour le traitement d'images sur un système UNIX. L'objectif est de décharger le serveur principal en déléguant les calculs lourds (filtres graphiques) à des processus "Workers" éphémères. Le système repose sur une communication inter-processus (IPC) robuste combinant mémoire partagée, sémaphores nommés et tubes (FIFOs). Ce rapport détaille la conception de l'architecture, la gestion de la concurrence, ainsi que l'optimisation via le multi-threading et l'intégration de la librairie `stb_image`.

Table des matières

1	Introduction	3
2	Architecture Technique	3
2.1	Vue d'ensemble	3
2.2	Communication Client $\leftrightarrow$ Serveur	3
2.3	Communication Worker $\rightarrow$ Client	3
3	Implémentation du Worker	3
3.1	Multithreading (Pthreads)	4
3.2	Gestion des Formats d'Images (Bonus)	4
4	Robustesse et Sécurité	4
4.1	Gestion du Timeout (Client)	4
4.2	Nettoyage des Ressources	5
5	Conclusion	5

1 Introduction

Le traitement d'image est une opération coûteuse en ressources CPU. Dans un système multi-utilisateurs classique, traiter ces requêtes de manière séquentielle sur un serveur unique créerait un goulot d'étranglement.

Notre solution propose une approche asynchrone :

1. Le client dépose une demande et se met en attente.
2. Le serveur orchestre la création de travailleurs (Workers).
3. Les workers effectuent le travail en parallèle et répondent directement au client.

2 Architecture Technique

2.1 Vue d'ensemble

Le système suit le modèle **Producteur-Consommateur** :

- **Producteur (Client)** : Génère des requêtes de traitement.
- **Tampon (SHM)** : Zone mémoire partagée stockant une file d'attente circulaire.
- **Consommateur (Serveur)** : Récupère les requêtes et lance les Workers.

2.2 Communication Client ↔ Serveur

La communication initiale passe par un segment de mémoire partagée (`shm_open`). Pour garantir l'intégrité des données, nous utilisons trois sémaphores nommés :

- `SEM_MUTEX` : Exclusion mutuelle pour modifier les index du tampon.
- `SEM_EMPTY` : Compte les places libres (bloque le client si tampon plein).
- `SEM_FULL` : Compte les requêtes en attente (réveille le serveur).

```
1 struct filter_request {  
2     pid_t pid;  
3     char chemin[MAX_PATH_LENGTH];  
4     int filtre;  
5     int parametres[5];  
6 };
```

Listing 1 – Structure de la requête en mémoire partagée (common.h)

2.3 Communication Worker → Client

Pour éviter de surcharger le serveur, le retour de l'image traitée ne repasse pas par lui. Le Worker établit un canal direct avec le Client via un **Tube Nommé (FIFO)**. Le nom du tube est conventionné : `/tmp/fifo_img_<PID_CLIENT>`.

3 Implémentation du Worker

Le Worker est le cœur de calcul du projet. Il a été conçu pour être performant et modulaire.

3.1 Multithreading (Pthreads)

Plutôt que de traiter l'image pixel par pixel séquentiellement, le Worker divise l'image en bandes horizontales. Chaque bande est traitée par un thread distinct.

```
1 int rows_per_thread = shared_img->height / nb_threads;
2
3 for (int i = 0; i < nb_threads; i++) {
4     workspace[i].ligne_debut = i * rows_per_thread;
5     workspace[i].ligne_fin = (i == nb_threads - 1)
6                             ? shared_img->height
7                             : (i + 1) * rows_per_thread;
8
9     pthread_create(&threads[i], NULL, worker_routine_thread, &workspace[
10 i]);
11 }
```

Listing 2 – Routine de lancement des threads (worker.c)

Cette approche "Zero-Copy" est efficace : l'image est chargée une seule fois en mémoire (heap), et tous les threads lisent/écrivent directement dans ce buffer partagé sans duplication de données.

3.2 Gestion des Formats d'Images (Bonus)

Initialement, le projet gérait manuellement les fichiers BMP. Cependant, la gestion du *padding* et des en-têtes complexes s'est avérée instable.

Nous avons intégré la bibliothèque **stb_image** (fichiers `img.c` et `img.h`). Cela apporte plusieurs avantages majeurs :

- Support des formats compressés : JPEG, PNG, BMP, TGA.
- Robustesse : Gestion automatique des allocations mémoire et des erreurs de format.
- Conservation du format : Le système accepte n'importe quelle entrée et génère une image de sortie du même type que l'original.

4 Robustesse et Sécurité

Un système distribué doit être résilient aux pannes.

4.1 Gestion du Timeout (Client)

Si un Worker plante, le client ne doit pas rester bloqué indéfiniment en attente d'une ouverture de tube. Nous avons implémenté un mécanisme de timeout avec `alarm()` et `SIGALRM`.

```
1 struct sigaction sa;
2 sa.sa_handler = handle_alarm;
3 sigaction(SIGALRM, &sa, NULL);
4
5 alarm(10);
6 int fifo_fd = open(fifo_name, O_RDONLY);
7 alarm(0);
8
9 if (fifo_fd == -1 && errno == EINTR) {
10     fprintf(stderr, "Erreur: Le serveur ne repond pas (Timeout).\n");
11 }
```

```
11     exit(EXIT_FAILURE);  
12 }
```

Listing 3 – Protection contre le blocage infini (client.c)

4.2 Nettoyage des Ressources

Le serveur intercepte les signaux pour garantir la propreté du système :

- **SIGINT (Ctrl+C)** : Suppression immédiate des segments de mémoire partagée (`shm_unlink`) et des sémaphores pour ne pas polluer le dossier `/dev/shm/`.
- **SIGCHLD** : Appel à `waitpid` avec l'option `WNOHANG` pour éliminer les processus zombies (workers terminés) sans bloquer la boucle principale du serveur.

5 Conclusion

Ce projet nous a permis de maîtriser les concepts fondamentaux des Systèmes d'Exploitation : la synchronisation inter-processus, la gestion de la mémoire et le parallélisme.

L'architecture finale est fonctionnelle et robuste. L'intégration de `stb_image` rend l'application utilisable avec des images réelles (photos JPEG), et le multithreading assure des temps de réponse rapides même sur des fichiers volumineux.